



PHASE 3 — PROJECT DOCUMENTATION



Maintenance Request System

ASP.NET Core 10 · MVC + REST API

ASP.NET Core 10

Razor MVC

REST API

Entity Framework Core

SQL Server 2025

JWT + Identity

Swagger / OpenAPI

DEVELOPMENT TEAM

- 1 Abdallah Waleed Kamal Mousa Salem
- 2 Saif Sameh Fathy Elsayy
- 3 Abdel Rahman Khaled Refaat Hussein
- 4 Mazen Ali Abdel Halim Baza
- 5 Amina Essam Mohamed Kamel Youssef



NAVIGATION

Table of Contents

01	Project Overview & Objectives	3
02	Technology Stack	4
03	System Architecture & Project Structure	5
04	Data Models & Database Design	6
05	MVC Web Application — Controllers & Views	7
06	REST API — Endpoints & Design	9
07	Security & Authorization	11
08	Application Screenshots	12
09	Roles & Permissions Matrix	16
10	Conclusion & Team Contributions	17

 **GitHub Repository:**
<https://github.com/saifsamehelsawy/Maintenance-Request-System>

CHAPTER 01

Project Overview & Objectives

The **Maintenance Request System** is a full-featured enterprise web application developed under the ITI .NET Training Program (Phase 3). It digitizes the entire lifecycle of maintenance requests — from initial employee submission, through admin assignment and technician work, to final closure — replacing fragmented manual processes with a unified, role-aware digital platform.

1.1 Problem Statement

Without a dedicated system, organizations rely on emails, phone calls, and paper forms to report and track facility issues. This results in lost requests, no accountability, no history, and poor response times. This project solves those problems by providing a unified, traceable, and role-controlled digital workflow.

1.2 Core Objectives

 Full Lifecycle Submit → Assign → Work → Resolve → Close with audit trail	 Role-Based Access Admin, Technician, Employee with distinct scoped permissions	 Dual Interface Razor MVC for browsers + REST API for mobile/external apps
 Enterprise Security JWT, Cookie Auth, rate limiting, soft-delete, global handler	 Role Dashboards Real-time statistics and recent activity per role	 API Docs Interactive Swagger / OpenAPI for all 30+ REST endpoints

1.3 Project Metrics

5 MVC Controllers	5 API Controllers	30+ REST Endpoints	3 User Roles	5 Status States	4 Priority Levels
-----------------------------	-----------------------------	------------------------------	------------------------	---------------------------	-----------------------------

1.4 Request Status Lifecycle

Pending → (Admin assigns) → **Assigned** → (Technician starts) → **In Progress** → (Technician resolves) → **Resolved**
→ (Admin/Employee closes) → **Closed**

Every state transition is recorded automatically in the immutable audit history log.

CHAPTER 02

Technology Stack

Layer	Technology	Version	Purpose
Framework	ASP.NET Core	10.0 LTS	Application host and HTTP pipeline
Web UI	Razor MVC + Bootstrap	5.3	Server-rendered responsive HTML pages
REST API	ASP.NET Core Web API	10.0	JSON REST endpoints for external consumers
ORM	Entity Framework Core	10.0	Database abstraction, LINQ, code-first migrations
Database	SQL Server	2025 Express	Relational data persistence
Identity	ASP.NET Core Identity	10.0	User management, password hashing, roles
API Auth	JWT Bearer	—	Stateless token authentication for REST API
API Docs	Swagger / Swashbuckle	9.x	Interactive OpenAPI 3.0 documentation UI
Rate Limiting	ASP.NET Core Limiter	10.0	Brute-force protection on auth endpoints
Logging	Microsoft.Extensions.Logging	10.0	Structured application-wide logging

2.1 Key NuGet Packages

Package	Purpose
<code>Microsoft.AspNetCore.Identity.EntityFrameworkCore</code>	Identity with EF Core integration
<code>Microsoft.EntityFrameworkCore.SqlServer</code>	SQL Server database provider for EF Core
<code>Microsoft.AspNetCore.Authentication.JwtBearer</code>	JWT Bearer token middleware
<code>Swashbuckle.AspNetCore</code>	Swagger UI and OpenAPI JSON generation
<code>System.IdentityModel.Tokens.Jwt</code>	JWT token creation and signature validation

2.2 Development Tooling

Visual Studio 2022 / VS Code

.NET SDK + EF Core CLI

SQL Server 2025 Express

Git & GitHub for source control

Postman for API endpoint testing

Bootstrap 5 for responsive frontend

Swagger UI for interactive API docs

Cytoscape headless for PDF generation

CHAPTER 03

System Architecture & Project Structure

The system follows a **three-tier layered architecture**. Both the MVC web layer and the REST API share the same *Services Layer* — no duplication, consistent behaviour.



3.1 Folder Structure

Folder / File	Role	Description
Controllers/	MVC	Handle browser requests, return Razor Views
Controllers/API/	REST API	Handle API requests, return JSON payloads
Models/	Entities + Enums	Domain objects mapped to SQL Server tables
DTOs/	Data Transfer	Input/output shapes for API validation
ViewModels/	View Models	Strongly-typed data from controller to Razor view
Services/	Business Logic	Shared by both MVC and REST API layers
Data/	DB Context	EF Core ApplicationDbContext + Migrations
Middleware/	Cross-cutting	Global exception handler (JSON/page by context)
Extensions/	DI Config	JWT, Identity, Swagger, CORS, Rate Limiter setup
Helpers/	Utilities	Request number generator REQ-YYYY-XXXX

CHAPTER 04

Data Models & Database Design

Entity	Key Fields	Relationships
ApplicationUser	Id, FullName, Email, IsActive, CreatedAt	Extends IdentityUser · 1-to-many with Requests (Employee & Technician)
MaintenanceRequest	Id, RequestNumber, Title, Description, Status, Priority, CreatedAt, ResolvedAt, IsDeleted	FK to Employee, Technician, Category · Has Comments, History
Category	Id, Name, Description, IsActive	One-to-many with MaintenanceRequest
RequestComment	Id, Content, CreatedAt, IsDeleted	Belongs to Request and ApplicationUser
RequestHistory	Id, Action, OldValue, NewValue, ChangedAt	Immutable audit log per Request + User

4.1 Enumerations

Request Status

-  Pending — Submitted, awaiting assignment
-  Assigned — Technician assigned by Admin
-  In Progress — Technician actively working
-  Resolved — Technician marked as fixed
-  Closed — Formally closed

Priority Level

-  Low — Minor, non-urgent
-  Medium — Moderate impact
-  High — Significant impact
-  Critical — Business-stopping, urgent

4.2 Seed Data & Request Numbers

Auto-Seed on First Run: 5 Categories (Electrical, Plumbing, HVAC, IT Equipment, Furniture) · 3 Roles (Admin, Technician, Employee) · 3 Demo Users

Request Number Format: `REQ-{YEAR}-{SEQUENCE:0000}` e.g. **REQ-2026-0001**

CHAPTER 05

MVC Web Application

The MVC layer is a full server-rendered web app built with **ASP.NET Core Razor Views** and **Bootstrap 5**, using **Cookie Authentication**. Unauthenticated requests are redirected to `/Account/Login`.



MVC Data Flow Razor MVC

Route → Controller Action → Service Layer → ViewModel → Razor View → HTML Response. Views never access the database directly.

AccountController Public

GET/POST /Account/Login — Sign-in form, validates credentials via Identity, issues auth cookie

GET/POST /Account/Register — Registration form, creates user, assigns Employee role

POST /Account/Logout — Signs out, clears cookie, redirects to login

DashboardController Authenticated

Admin Dashboard — Total/pending/in-progress/resolved/critical requests, users, technicians, recent 5

Technician Dashboard — Assigned to me, in-progress, resolved this month

Employee Dashboard — My total, pending, in-progress, resolved, closed

RequestsController Authenticated

GET /Requests — Paginated list with Status, Priority, Category, keyword, date filters + sortable columns

GET /Requests/Details/{id} — Full request: metadata, status badge, comment thread, audit history timeline

GET/POST /Requests/Create — Employee creates request; auto-generates REQ-YYYY-XXXX number

GET/POST /Requests/Edit/{id} — Employee/Admin edits Pending requests only

POST /Requests/Delete/{id} — Soft-delete; Employee: own; Admin: any

POST /Requests/Assign/{id} — Admin assigns technician from active-technician dropdown

POST /Requests/ChangeStatus/{id} — Technician/Admin changes status with optional resolution notes

POST /Requests/AddComment/{id} — Any authenticated user adds comment

CategoriesController Admin Only

GET /Categories — Lists all categories with request counts

GET/POST /Categories/Create — Create new category with name and description

GET/POST /Categories/Edit/{id} — Update category name / description / active flag

POST /Categories/Delete/{id} — Soft-delete (only if no active requests)

UsersController Admin Only

GET /Users — Paginated user list with role badge, active/inactive status

GET /Users/Details/{id} — Full profile with request history

GET/POST /Users/Edit/{id} — Edit full name, role, active flag

POST /Users/Activate/{id} — Activates a deactivated user account

POST /Users/Deactivate/{id} — Prevents login without deleting account

5.1 Views Summary

Folder	Views	Key Features
Account/	Login, Register, AccessDenied	Demo account hints, validation summaries
Dashboard/	Admin, Technician, Employee	Color-coded stat cards, recent request table
Requests/	Index, Details, Create, Edit	Filters, audit timeline, comment thread, status update
Categories/	Index, Create, Edit	Request count per category
Users/	Index, Details, Edit	Role badges, activate/deactivate buttons
Shared/	_Layout.cshtml	Role-aware sidebar, API Docs link for Admin only

Shared Layout Sidebar: `_Layout.cshtml` uses `User.IsInRole("...")` to dynamically show/hide navigation. Admin sees: Dashboard · Requests · Categories · Users · API Docs. Technician and Employee see only what is relevant to their workflow.

CHAPTER 06

REST API — Endpoints & Design

Complete REST API under `/api/v1`. Protected endpoints require `Authorization: Bearer {token}`. Tokens expire after **60 minutes**. Interactive docs at `/swagger`.

trialsAll responses use a standard envelope: `{ success, message, data }`

Validation errors return field-level details: `{ field, message }[]`

Paginazione via `page`, `pageSize` query params (max 100)

- Soft delete — `IsDeleted` flag, no record permanently erased

Rate limit: 5 auth requests/minute — returns HTTP 429 on excess

6.1 Auth — /api/v1/auth

POST /api/v1/auth/register — Register new account (public)

POST /api/v1/auth/login – Authenticate, receive signed JWT (public)

POST /api/v1/auth/logout – Invalidate current session (authenticated)

6.2 Requests — /api/v1/requests

GET /api/v1/requests — Paginated list: status, priority, category, search, date, sort

POST /api/v1/requests — Create new request (Employee / Admin)

GET /api/v1/requests/{id} — Full details of a request

PUT /api/v1/requests/{id} — Update title/description (Employee own / Admin any)

DELETE /api/v1/requests/{id} – Soft-delete request

POST /api/v1/requests/{id}/assign — Assign technician (Admin only)

PUT /api/v1/requests/{id}/status — Change status + resolution notes (Technician/Admin)

GET /api/v1/requests/{id}/comments — Get all comments

POST `/api/v1/requests/{id}/comments` — Add comment to request thread

GET /api/v1/requests/{id}/history — Full immutable audit history

6.3 Categories — `/api/v1/categories`

GET	/api/v1/categories	— All active categories (All roles)
GET	/api/v1/categories/{id}	— Single category by ID
POST	/api/v1/categories	— Create category (Admin)
PUT	/api/v1/categories/{id}	— Update category (Admin)
DELETE	/api/v1/categories/{id}	— Delete category (Admin)

6.4 Users — `/api/v1/users`

GET	/api/v1/users	— Paginated user list (Admin)
GET	/api/v1/users/{id}	— User profile (Admin)
PUT	/api/v1/users/{id}	— Update user name / role (Admin)
PUT	/api/v1/users/{id}/activate	— Activate user (Admin)
PUT	/api/v1/users/{id}/deactivate	— Deactivate user (Admin)
GET	/api/v1/users/technicians	— All active technicians (Admin)

6.5 Dashboard — `/api/v1/dashboard`

GET	/api/v1/dashboard/admin	— Full admin statistics (Admin)
GET	/api/v1/dashboard/technician	— Technician workload stats
GET	/api/v1/dashboard/employee	— Employee request summary

6.6 Standard Response Envelope

```
// ✅ Success
{
  "success": true,
  "message": "OK",
  "data": { ... }
}
```

```
// ❌ Validation Error
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    { "field": "Title",
      "message": "Required" }
  ]
}
```

Swagger UI: Navigate to `/swagger` · Click *Authorize* · Paste `Bearer {token}` · Test any protected endpoint interactively.

CHAPTER 07

Security & Authorization

MVC — Cookie Auth

After login via `/Account/Login`, Identity issues an encrypted session cookie sent with every page request. On logout the cookie is revoked.

API — JWT Bearer

Client posts to `POST /api/v1/auth/login`, receives a signed JWT. Included as `Authorization: Bearer {token}` on all protected requests. Expires after **60 min**.

7.1 Security Features

- JWT Bearer Authentication with 60-minute expiry for stateless API access
- Cookie-based session authentication for stateful browser navigation
- Role-based Authorization: `[Authorize(Roles="Admin")]` on controller actions
- Role-level Authorization: Employees can only access their own requests
- Rate Limiting: max **5 login attempts per minute** per IP — returns HTTP 429
- Soft Delete: `IsDeleted` flag — no record permanently erased
- Password hashing: PBKDF2 with HMACSHA256 via ASP.NET Core Identity
- Passwords are **never** included in any API response payload
- Global Exception Middleware: hides stack traces; structured JSON for API, error page for MVC
- CORS: explicitly configured allowed origins only
- Input Validation: Data Annotations on all DTOs enforced at model-binding

7.2 JWT Configuration

⚠ Production: Never store JWT secret in `appsettings.json` in source control. Use environment variables or Azure Key Vault.

```
{
  "Jwt": {
    "Key": "<minimum-32-character-random-secret>",
    "Issuer": "MaintenanceRequestSystem",
    "Audience": "MaintenanceRequestSystemUsers",
    "ExpirationMinutes": "60"
  },
  "RateLimiting": { "LoginWindowMinutes": "1", "LoginMaxAttempts": "5" },
  "Cors": { "AllowedOrigins": ["http://localhost:3000"] }
}
```

CHAPTER 08

Application Screenshots

All screenshots captured from the live running application.

Fig 1 — Login Page

Sign In

Email Address

Enter your email

Password

Enter your password

Remember me

Sign In

Don't have an account? Register

Demo Accounts:

Admin admin@maintenance.com / Admin@123

Tech tech@maintenance.com / Tech@123

Emp emp@maintenance.com / Emp@123

Fig 2 — User Registration

Create Account

Full Name

Your full name

Email

you@example.com

Department

Optional

Password

Min 6 characters

Confirm Password

Min 6 characters

Role

Employee

Register

Already have an account? Sign In

Fig 3 — Admin Dashboard — Statistics Overview & Recent Requests

admin@maintenance.com

Admin

API Docs

Logout

Dashboard

Requests

Categories

Users

Admin Dashboard

Saturday, September 5, 2026

3 Total Requests

2 Pending

0 In Progress

1 Resolved

0 Closed

0 Critical

4 Total Users

1 Technicians

Recent Requests

View All

Request #	Title	Status	Priority	Employee	Created	
REQ-2026-0003	AC not cooling	Pending	Medium	Sara Mohamed	Sep 5, 2026	
REQ-2026-0002	Printer paper jam	Resolved	High	Sara Mohamed	Sep 5, 2026	
REQ-2026-0001	Laptop not turning on	Pending	High	Sara Mohamed	Sep 5, 2026	

Fig 4 — Maintenance Requests List — Filters, Search & Status Badges

 Maintenance Request System

admin@maintenance.com Admin

[API Docs](#)

[Logout](#)

Dashboard

Requests

Categories

Users

Maintenance Requests

All Status 

All Priority 

All Categories 





Showing 1 - 3 of 3 requests

Request #	Title	Status	Priority	Category	Employee	Technician	Created	
REQ-2026-0003	AC not cooling	Pending	Medium	HVAC	Sara Mohamed	-	Sep 5, 2026	
REQ-2026-0002	Printer paper jam	Resolved	High	IT Equipment	Sara Mohamed	Ahmed Hassan	Sep 5, 2026	
REQ-2026-0001	Laptop not turning on	Pending	High	IT Equipment	Sara Mohamed	-	Sep 5, 2026	

Fig 5 — Request Details — Status, Comments & Audit History Timeline

✕ Maintenance Request System

admin@maintenance.comAdminAPI DocsLogout

Dashboard

Requests

Categories

Users

REQ-2026-0002

Created September 5, 2026 at 07:22

DeleteBack

Request Details

Printer paper jam

Office printer has constant paper jam issues

STATUS

Resolved

PRIORITY

High

CATEGORY

IT Equipment

EMPLOYEE

Sara Mohamed

TECHNICIAN

Ahmed Hassan

RESOLVED AT

Sep 5, 2026

RESOLUTION NOTES

Cleared the paper jam and recalibrated the rollers.

Change Status

Pending

Resolution notes (optional)

Update

Comments (1)

Ahmed Hassan Sep 5, 2026 07:22

I have opened the printer and found the jam.

Add a comment

History

RequestCreated

Maintenance request created

Sara Mohamed — Sep 5, 07:22

TechnicianAssigned

Technician assigned to request

System Administrator — Sep 5, 07:22

StatusChanged

Status changed to InProgress

Ahmed Hassan — Sep 5, 07:22

CommentAdded

Comment added

Ahmed Hassan — Sep 5, 07:22

StatusChanged

Status changed to Resolved

Ahmed Hassan — Sep 5, 07:22

Maintenance Request System — Documentation · ITI .NET Track 2025–2027

Page 13

Fig 6 – Categories Management

Name	Description	Requests	Active	Created
Electrical	Electrical systems and wiring issues	4	Active	Jan 1, 2025
Furniture	Furniture repair and replacement	0	Active	Jan 1, 2025
HVAC	Heating, ventilation, and air conditioning	1	Active	Jan 1, 2025
IT Equipment	Computers, printers, and network equipment	2	Active	Jan 1, 2025
Plumbing	Water and drainage issues	0	Active	Jan 1, 2025

Fig 7 – Users Management Panel

Name	Email	Department	Roles	Requests	Status	Actions
Ahmed Hassan	tech@maintenance.com	Maintenance	Technician	0	Active	[Edit] [Delete]
Ahmed Tector	emp2@maintenance.com	-	Employee	0	Active	[Edit] [Delete]
Sara Mohamed	emp@maintenance.com	HR	Employee	3	Active	[Edit] [Delete]
System Administrator	admin@maintenance.com	IT	Admin	0	Active	[Edit] [Delete]

Fig 8 – Employee Dashboard

Request #	Title	Status	Priority	Created
REQ-2025-0003	AC not cooling	Pending	Medium	Sep 5, 2025
REQ-2025-0002	Printer paper jam	Resolved	High	Sep 5, 2025
REQ-2025-0001	Laptop not turning on	Pending	High	Sep 5, 2025

Fig 9 – Technician Dashboard

Request #	Title	Status	Priority	Employee
No active requests assigned to you				

Fig 10 – Create New Request (Employee)

New Maintenance Request

Title *
Brief description of the issue

Description *
Detailed description of the issue...

Category *
Select category...

Priority *
Low

[Submit Request] [Cancel]

Fig 11 – API JSON Response

```

{
  "success": true,
  "message": "Request retrieved successfully",
  "data": {
    "id": 2,
    "requestNumber": "REQ-2025-0002",
    "title": "Printer paper jam",
    "description": "Office printer has constant paper jam issues.",
    "status": "Resolved",
    "priority": "High",
    "resolutionNotes": "I cleared the paper jam and recalibrated the rollers.",
    "createdAt": "2025-09-05T07:22:46.1158425",
    "updatedAt": "2025-09-05T07:22:47.6596608",
    "resolvedAt": "2025-09-05T07:22:47.6596608",
    "closedAt": null,
    "category": {
      "id": 4,
      "name": "IT Equipment"
    },
    "employee": {
      "id": "b072d5c-990b-4c30-811d-003ff55d143",
      "fullName": "Sara Mohamed",
      "email": "emp@maintenance.com",
      "department": "HR"
    },
    "technician": {
      "id": "9190e0e2-d837-4a7f-9103-5c8f90e42f6",
      "fullName": "Ahmed Hassan",
      "email": "tech@maintenance.com",
      "department": "Maintenance"
    },
    "comments": [
      {
        "id": 1,
        "commentText": "I have opened the printer and found the jam.",
        "createdAt": "2025-09-05T07:22:47.6596608"
      }
    ]
  }
}

```

Fig 12 — Swagger / OpenAPI Interactive Documentation

Swagger
Supported by SMARTBEAR

Select a definition Maintenance Request System API v1

Maintenance Request System API ^{v1} OAS 3.0

/swagger/v1/swagger.json

REST API for Maintenance Request System - supports MVC Web UI and external clients

Authorize

AuthApi

- POST /api/v1/auth/register
- POST /api/v1/auth/login
- POST /api/v1/auth/logout

CategoriesApi

- GET /api/v1/categories
- POST /api/v1/categories
- GET /api/v1/categories/{id}
- PUT /api/v1/categories/{id}
- DELETE /api/v1/categories/{id}

Fig 13 — Swagger — Requests Endpoints Expanded

 **Swagger**
supported by SMARTBEAR

Select a definition **Maintenance Request System API v1**

Maintenance Request System API v1 OAS 3.0

/swagger/v1/swagger.json

REST API for Maintenance Request System - supports MVC Web UI and external clients

Authorize 

AuthApi

POST

/api/v1/auth/register





POST

/api/v1/auth/login





POST

/api/v1/auth/logout





CategoriesApi

GET

/api/v1/categories





POST

/api/v1/categories





GET

/api/v1/categories/{id}





PUT

/api/v1/categories/{id}





DELETE

/api/v1/categories/{id}





DashboardApi

GET

/api/v1/dashboard/admin





GET

/api/v1/dashboard/technician





GET

/api/v1/dashboard/employee





RequestsApi

Maintenance Request System — Documentation · ITI .NET Track 2025–2027

Page **15**

CHAPTER 09

Roles & Permissions Matrix

Feature / Action	Employee	Technician	Admin
View requests	✓ Own only	✓ Assigned	✓ All
Create request	✓	✗	✓
Edit Pending request	✓ Own	✗	✓ Any
Delete Pending request	✓ Own	✗	✓ Any
Assign technician	✗	✗	✓
Change request status	✗	✓ Own assigned	✓
Add comments	✓	✓	✓
View audit history	✓ Own	✓ Assigned	✓ All
Manage categories	✗	✗	✓
Manage users	✗	✗	✓
Admin Dashboard	✗	✗	✓
Technician Dashboard	✗	✓	✗
Employee Dashboard	✓	✗	✗
Swagger / API Docs	✗	✗	✓

9.1 Demo Credentials

Role	Email	Password
Admin	admin@maintenance.com	Admin@123
Technician	tech@maintenance.com	Tech@123
Employee	emp@maintenance.com	Emp@123

9.2 Quick Start

- 1 Install .NET 10 SDK and SQL Server 2025 Express
- 2 Run `dotnet ef database update`
- 3 Run `dotnet run`
- 4 Open `https://localhost:5001` — MVC Web UI
- 5 Open `https://localhost:5001/swagger` — API Docs

CHAPTER 10

Conclusion & Team Contributions

10.1 Summary

The Maintenance Request System is a production-ready enterprise application demonstrating full mastery of ASP.NET Core 10. It delivers a **Razor MVC web interface** and a **complete REST API** from a single shared codebase, powered by a common services layer ensuring consistency and eliminating duplication. The project follows clean layered architecture, separation of concerns, DTO-based API design, and resource-level authorization.

10.2 Key Achievements

- Single unified interface — Razor MVC + REST API — from a single shared business logic codebase
- Full request lifecycle with timestamped, immutable audit history on every state change
- Cookie Auth (MVC) and JWT Bearer (API) seamlessly integrated and correctly separated
- Swagger/OpenAPI interactive documentation covering all **30+ endpoints**
- Role-based and resource-level authorization enforced at the service layer
- Automatic database seeding — immediately usable on first run
- Global exception handler with environment-aware responses (JSON / error page)
- Rate limiting on auth endpoints — brute-force protection built in

10.3 Development Team

#	Full Name	Primary Contribution
1	Abdallah Waleed Kamal Mousa Salem	Architecture, MVC Controllers, Services Layer, Cookie Auth
2	Saif Sameh Fathy Elsayy	REST API Controllers, JWT Integration, Swagger Configuration
3	Abdel Rahman Khaled Refaat Hussein	Data Models, EF Core Migrations, Database Design & Seeding
4	Mazen Ali Abdel Halim Baza	Razor Views, Bootstrap UI, Dashboard Design, Frontend Logic
5	Amina Essam Mohamed Kamel Youssef	DTOs, ViewModels, Input Validation, Testing & QA

Information Technology Institute (ITI)

.NET Track · Academic Year 2025–2027

 github.com/saifsamehelsawy/Maintenance-Request-System